



Name : Deras Pangestu Gusti (08)
 Class : SIB 1G
 NIM : 254107060114

JOBSHEET

Double Linked Lists

12.1 Learning Objectives

After completing this practical session, students will be able to:

1. Understand the double linked list algorithm.
2. Create and declare the structure of a double linked list algorithm.
3. Apply the double linked list algorithm in various case studies.

12.2 Experiment 1

12.2.1 Activity 1

In Experiment 1, a **Student** class, a **Node** class, and a **DoubleLinkedList** class will be created. These classes will include operations for adding data in multiple ways (from the front and the back of the linked list).

1. Observe the class diagram of Student, Node, and DoubleLinkedLists below. This class diagram will serve as a reference for writing the DoubleLinkedLists program code.

Student
nim: String name: String className: String gpa: Double
Student(String nim, String name, String className, Double gpa) print(): void

Node
data: Student prev: Node next: Node
Node() Node(data: Student) Node(prev: Node, data: Student, next: Node)



DoubleLinkedLists
head: Node tail : Node
DoubleLinkedLists() isEmpty(): boolean addFirst (): void addLast(): void add(item: int, index:int): void print(): void removeFirst(): void removeLast(): void insertAfter: void

2. Create a new package named **P13** or by using your preferred name
3. Create a class within the package named **Student**. In this class, declare the attributes according to the class diagram above. Also, add the constructor and methods as specified in the diagram.

```
package P13;

public class Student {
    String nim, name, className;
    double gpa;

    public Student(){
    }
    public Student(String nm, String nama, String kls, double ip){
        nim = nm;
        name = nama;
        className = kls;
        gpa = ip;
    }
    void print(){
        System.out.println(nim+" - "+name+" - "+className+" - "+gpa);
    }
}
```

4. Create a class within the package named **Node**. In this class, declare the attributes according to the class diagram above. Then, add the constructor as specified in the diagram.

```
public class Node {
```

```

    Student data;
    Node prev;
    Node next;

    Node(){
    }
    Node(Student data){
        this.data = data;
        prev = null;
        next = null;
    }
    Node(Node prev, Student data, Node next){
        this.data = data;
        this.prev = prev;
        this.next = next;
    }
}

```

5. Create a new class named **DoubleLinkedLists** in the same package as **Node**. In the **DoubleLinkedLists** class, declare the attributes according to the class diagram above.

```

public class DoubleLinkedLists {
    Node head;
    Node tail;
}

```

6. Add constructor to initialize **head** and **tail** attributes to be **null**;

```

DoubleLinkedLists(){
    head = null;
    tail = null;
}

```

7. Create a new method **isEmpty()** that will return true in case the double linked list is still empty.

```

boolean isEmpty(){
    return head==null;
}

```

8. Next, create the **addFirst()** method. This method will perform data insertion at the front of the linked list.

```

void addFirst(Student data){
    Node newNode = new Node(data);
    if(isEmpty()){
        head = tail = newNode;
    }else{

```



```

        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}

```

9. In addition, create the **addLast()** method to insert data at the end of the linked list.

```

void addLast(Student data){
    Node newNode = new Node(data);
    if(isEmpty()){
        head = tail = newNode;
    }else{
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}

```

10. To insert data after the node that contains the specified key, you can do it as follows.

```

void insertAfter(String key, Student data){
    Node newNode = new Node(data);
    Node temp = head;
    while(temp!=null){
        if(temp.data.nim.equalsIgnoreCase(key)){
            if(temp == tail){
                addLast(data);
            }else{
                newNode.next = temp.next;
                newNode.prev = temp;
                temp.next.prev = newNode;
                temp.next = newNode;
            }
        }
        temp = temp.next;
    }
    if(temp == null){
        System.out.println("Insertion failed. Data (" +key+") not found!!");
    }
}

```

11. To display the contents of the linked list, create a **print()** method. This method will print all elements in the linked list, regardless of its size.

```

void print(){
    if(!isEmpty()){
        Node temp = head;
        while(temp!=null){

```

```

        temp.data.print();
        temp = temp.next;
    }
    System.out.println("");
} else {
    System.out.println("Double linked list is currently
empty!!");
}
}

```

12. Next, create the **DoubleLinkedListsMain** class to execute all the methods available in the **DoubleLinkedLists** class.

```

public class DoubleLinkedListsMain {
    public static void main(String[] args) {
        DoubleLinkedLists dll = new DoubleLinkedLists();
        dll.print();
        dll.addFirst(new Student("111", "Anton", "TI-1I", 3.57));
        dll.print();
        dll.addLast(new Student("112", "Prabowo", "TI-1I", 3.7));
        dll.print();
        dll.addFirst(new Student("113", "Herco", "TI-1I", 3.89));
        dll.print();
        dll.insertAfter("111", new Student("114", "Rizki", "TI-1I",
3.8));
        dll.print();
        dll.insertAfter("112", new Student("115", "Hanzel", "TI-1I",
3.6));
        dll.print();
        dll.insertAfter("120", new Student("116", "Eiyu", "TI-1I",
3.4));
        dll.print();
    }
}

```

13. Run and observe the output!

```
1 package com.jobsheet13;
2
3 public class Student
4 {
5     String nim,
6         name,
7         className;
8     double gpa;
9
10    public Student() {}
11    public Student (String nm, String nama, String kls, double ip)
12    {
13        nim      = nm;
14        name     = nama;
15        className = kls;
16        gpa      = ip;
17    }
18
19    void print()
20    {
21        System.out.println(nim + " - " + name + " - " + className + " - " + gpa);
22    }
23 }
24
25 package com.jobsheet13;
26
27 public class Node
28 {
29     Student data;
30     Node prev, next;
31
32     public Node() {}
33     public Node (Student data)
34     {
35         this.data = data;
36         prev      = null;
37         next      = null;
38     }
39
40     public Node(Node prev, Student data, Node next)
41     {
42         this.data = data;
43         this.prev = prev;
44         this.next = next;
45     }
46 }
```

```
1  package com.jobsheet13;
2
3  public class DoubleLinkedLists
4  {
5      Node head, tail;
6
7      public DoubleLinkedLists()
8      {
9          head = null;
10         tail = null;
11     }
12
13     boolean isEmpty()
14     {
15         return head == null;
16     }
17
18     void addFirst (Student data)
19     {
20         Node newNode = new Node (data);
21
22         if (isEmpty())
23             head = tail = newNode;
24         else
25         {
26             newNode.next = head;
27             head.prev = newNode;
28             head = newNode;
29         }
30     }
31
32     void addLast (Student data)
33     {
34         Node newNode = new Node (data);
35
36         if (isEmpty())
37             head = tail = newNode;
38         else
39         {
40             tail.next = newNode;
41             newNode.prev = tail;
42             tail = newNode;
43         }
44     }
45 }
```

```

46 void insertAfter (String key, Student data)
47 {
48     Node newNode = new Node (data);
49     Node temp = head;
50
51     while (temp != null)
52     {
53         if (temp.data.nim.equalsIgnoreCase(key))
54         {
55             if (temp == tail)
56                 addLast(data);
57             else
58             {
59                 newNode.next = temp.next;
60                 newNode.prev = temp;
61                 temp.next.prev = newNode;
62                 temp.next = newNode;
63             }
64         }
65
66         temp = temp.next;
67     }
68
69     if (temp == null)
70         System.out.println("Insertion failed. Data (" + key + ") not found!!");
71 }
72
73 void print()
74 {
75     if (!isEmpty())
76     {
77         Node temp = head;
78
79         while (temp != null)
80         {
81             temp.data.print();
82             temp = temp.next;
83         }
84
85         System.out.println(x: "");
86     }
87     else
88         System.out.println(x: "Double linked list is currently empty!!");
89 }
90 }
91
92 package com.jobsheet13;
93
94 public class DoubleLinkedListMain
95 {
96     Run main | Debug main | Run | Debug
97     public static void main(String[] args)
98     {
99         DoubleLinkedLists dll = new DoubleLinkedLists();
100
101         dll.print();
102         dll.addFirst(new Student(nm: "111", nama: "Anton", kls: "TI-1I", ip: 3.57));
103         dll.print();
104         dll.addLast(new Student(nm: "112", nama: "Prabowo", kls: "TI-1I", ip: 3.7));
105         dll.print();
106         dll.addFirst(new Student(nm: "113", nama: "Herco", kls: "TI-1I", ip: 3.89));
107         dll.print();
108         dll.insertAfter(key: "111", new Student(nm: "114", nama: "Rizki", kls: "TI-1I", ip: 3.8));
109         dll.print();
110         dll.insertAfter(key: "112", new Student(nm: "115", nama: "Hanzel", kls: "TI-1I", ip: 3.6));
111         dll.print();
112         dll.insertAfter(key: "120", new Student(nm: "116", nama: "Eiyu", kls: "TI-1I", ip: 3.4));
113         dll.print();
114     }
115 }

```




```

Double linked list is currently empty!!
111 - Anton - TI-1I - 3.57

111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

Insertion failed. Data (111) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7

Insertion failed. Data (112) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

Insertion failed. Data (120) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

```

12.2.2 Output Verification

Verify your program's compilation results using the following.

```

Double linked list is currently empty!!
111 - Anton - TI-1I - 3.57

111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

Insertion failed. Data (111) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57

```

```

114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7

Insertion failed. Data (112) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

Insertion failed. Data (120) not found!!
113 - Herco - TI-1I - 3.89
111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

```

12.2.3 Questions

1. Explain the difference between a singly linked list and a doubly linked list.
A singly linked list only contains a forward pointer (next) in each node, allowing traversal in a single direction (from head to tail). Conversely, a doubly linked list contains both forward (next) and backward (prev) pointers within its node structure. This allows bidirectional traversal, meaning you can navigate both forwards and backwards through the list.
2. Observe the **Node** class, which contains the attributes **next** and **prev**. What are the purposes of these attributes?
 - a. *next: Holds the reference (memory address) of the succeeding node in the sequence.*
 - b. *prev: Holds the reference (memory address) of the preceding node in the sequence.**They maintain the structural integrity and allow two-way navigation across the nodes.*
3. Examine the constructor in the **DoubleLinkedLists** class. What is the purpose of this constructor?

```

DoubleLinkedLists(){
    head = null;
    tail = null;
}

```

The constructor initializes a newly created DoubleLinkedLists instance. It sets both the head and tail pointers to null, explicitly representing that the list is initially empty.

4. In the **addFirst()** method, what is the meaning of the following code?

```

if(isEmpty()){
    head = tail = newNode;
}

```

When inserting into an empty list (isEmpty() evaluates to true), the new node becomes both the initial node (head) and the final node (tail) of the list.

5. In the **addFirst()** method, what does the statement **head.prev = newNode** mean?
Before updating the head pointer to point to the new node, this statement establishes a backward link from the current first node back to the incoming newNode. This ensures that the double link remains unbroken when a node is added at the front.
6. In the **insertAfter()** method, what is the meaning of **current.next.prev = newNode**?
This updates the node that succeeds the current node (temp.next). It redirects its backward (prev) pointer to point to the newly inserted newNode instead of pointing back to temp.
7. In the experiment 1 code, in which method the traversal process implemented? What is the meaning of **temp = temp.next** ?

Traversal is implemented within the `print()` method and the `insertAfter()` method. `temp = temp.next` shifts the structural focus to the next node in the sequence, advancing the loop through the list until it reaches null.

8. In the `insertAfter()` method, what is the following code for?

```
if(temp == tail){
    addLast(data);
}
```

It checks if the matching key belongs to the final node in the list. If it does, it delegates the operation to `addLast(data)` to correctly update the tail pointer.

Do we have to implement it? What if we remove it?

Yes, it must be implemented. If removed, appending after the tail node would successfully link the new node forward, but the tail attribute of the `DoubleLinkedLists` class would erroneously continue pointing to the old tail instead of shifting to the new final node.

9. In the `insertAfter()` what is this statement `if(temp.data.nim.equalsIgnoreCase(key))` for?

Yes, it must be implemented. If removed, appending after the tail node would successfully link the new node forward, but the tail attribute of the `DoubleLinkedLists` class would erroneously continue pointing to the old tail instead of shifting to the new final node.

10. In the above output, after inserting data after 111 and 112, an **Insertion failed. Data (111) not found!!** warning is displayed. It should be no warning message, since key 111 is found in the list. Explain the reason why it happens? And correct the code!

In the provided code, even if a matching key is successfully located and a node is inserted, the loop does not stop. It proceeds to traverse until `temp` reaches the end and becomes null. Once the loop exits, the check `if(temp == null)` executes. Because `temp` will always end up as null after completing the full loop, the "Insertion failed" error message triggers unconditionally.

```
46 void insertAfter (String key, Student data)
47 {
48     Node newNode = new Node (data);
49     Node temp = head;
50
51     while (temp != null)
52     {
53         if (temp.data.nim.equalsIgnoreCase(key))
54         {
55             if (temp == tail)
56                 addLast(data);
57             else
58             {
59                 newNode.next = temp.next;
60                 newNode.prev = temp;
61                 temp.next.prev = newNode;
62                 temp.next = newNode;
63             }
64
65             return; // <-- Correct Code (Add return statement)
66         }
67
68         temp = temp.next;
69     }
70
71     if (temp == null)
72         System.out.println("Insertion failed. Data (" + key + ") not found!!");
73 }
```



Add a return; statement immediately after successfully completing the insertion inside the inner blocks so the method exits early.

12.3 Experiment 2

12.3.1 Labs Activities

In this **Experiment 2** several methods will be created to delete elements from the LinkedLists in the **DoubleLinkedLists** class. Deletion will be performed in three ways: at the front, at the end, and at a specified index within the linked list.

1. Create the **removeFirst()** method inside the **DoubleLinkedLists** class.

```
void removeFirst(){
    if(isEmpty()){
        System.out.println("Double linked list is currently
empty!!");
    }else if(head == tail){
        head = tail = null;
    }else{
        head = head.next;
        head.prev = null;
    }
}
```

2. Create the **removeLast()** method inside the **DoubleLinkedLists** class.

```
void removeLast(){
    if(isEmpty()){
        System.out.println("Double linked list is currently
empty!!");
    }else if(head == tail){
        head = tail = null;
    }else{
        tail = tail.prev;
        tail.next = null;
    }
}
```

3. Also, add the **remove(int index)** method to the **DoubleLinkedLists** class and observe the results.

```
void remove(int index){
    if(isEmpty()){
        System.out.println("Double linked list is currently
empty!!");
    }else if(index == 0){
        removeFirst();
    }else{
        Node temp = head;
        for(int i=0; i<index; i++){
            temp = temp.next;
        }
        if(temp == tail){
            removeLast();
        }else{
            temp.prev.next = temp.next;
        }
    }
}
```



```
        temp.next.prev = temp.prev;  
    }  
}
```

4. Reopen the **main** method in the **DoubleLinkedListsMain** class and add the following statements to delete data:

```
dll.removeFirst();  
dll.print();  
dll.removeLast();  
dll.print();  
dll.remove(1);  
dll.print();
```

5. Run and observe the output!

```
91     void removeFirst()
92     {
93         if (isEmpty())
94             System.out.println(x: "Double linked list is currently empty!!");
95         else if (head == tail)
96             head = tail = null;
97         else
98         {
99             head      = head.next;
100             head.prev  = null;
101         }
102     }
103
104     void removeLast()
105     {
106         if (isEmpty())
107             System.out.println(x: "Double linked list is currently empty!!");
108         else if (head == tail)
109             head = tail = null;
110         else
111         {
112             tail      = tail.prev;
113             tail.next  = null;
114         }
115     }
116
117     void remove (int index)
118     {
119         if (isEmpty())
120             System.out.println(x: "Double linked list is currently empty!!");
121         else if (index == 0)
122             removeFirst();
123         else
124         {
125             Node temp = head;
126
127             for (int i = 0; i < index; i++)
128                 temp = temp.next;
129
130             if (temp == tail)
131                 removeLast();
132             else
133             {
134                 temp.prev.next = temp.next;
135                 temp.next.prev = temp.prev;
136             }
137         }
138     }
139 }
```



```

23     dll.removeFirst();
24     dll.print();
25     dll.removeLast();
26     dll.print();
27     dll.remove(index: 1);
28     dll.print();
29 }
30 }

```

```

111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

```

```

111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7

```

```

111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

```

12.3.2 Output Verification

Verify your program's compilation results using the following.

```

111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7
115 - Hanzel - TI-1I - 3.6

111 - Anton - TI-1I - 3.57
114 - Rizki - TI-1I - 3.8
112 - Prabowo - TI-1I - 3.7

111 - Anton - TI-1I - 3.57
112 - Prabowo - TI-1I - 3.7

```

12.3.3 Questions

1. What is the use of the following statement in the `removeFirst()` method?

```

head = head.next;
head.prev = null;

```

- a. `head = head.next;` advances the head pointer to point to the second node in the sequence, making it the new first node.
 - b. `head.prev = null;` severs the link to the deleted old first node, freeing it for garbage collection and ensuring the new head has no preceding node.
2. Why is it important to include conditions and processes like the ones below in both `removeFirst()` and `removeLast()` methods? Explain!

```

else if(head == tail){
    head = tail = null;
}

```

This condition targets scenarios where the list contains exactly one node . It is critical because removing the sole remaining node means the list becomes entirely empty, requiring both head and tail pointers to be reset safely to null to prevent memory leaks or dangling

reference exceptions .

3. In the **removeLast()** method, if there is no **tail** attribute inside **DoubleLinkedLists** class, what approach or changes need to be made to the code of the method?

Without a tail reference, you must manually traverse the list from the head using a loop to find the final node.

```

106     void removeLast()
107     {
108         if(isEmpty())
109             System.out.println(x: "Double linked list is currently empty!!");
110         else if(head.next == null)
111             head = null;
112         else
113         {
114             Node temp = head;
115             while(temp.next != null)
116                 temp = temp.next;
117             temp.prev.next = null;
118         }
119     }
120 
```

4. What is the purpose of the initial **if(isEmpty())** check in the **remove(int index)** method?

It serves as a safety guard clause. If a deletion is attempted on an empty list, it prevents the program from throwing a `NullPointerException` when trying to access or manipulate null references, printing an error message instead.

5. Explain how the method handles the removal of a node at the beginning (**index == 0**) and at the end (**temp == tail**) of the linked list.
 - a. *At the beginning (index == 0): The method bypasses localized manual adjustments and directly forwards execution to the specialized `removeFirst()` method.*
 - b. *At the end (temp == tail): After tracking the index forward using a loop, if the referenced node matches the tail pointer, it routes execution directly to `removeLast()` to update the system boundaries properly.*
6. Describe how the method updates the links between nodes when removing a node from the middle of the linked list.



- a. *temp.prev.next = temp.next; tells the preceding node to point past temp and link directly to the succeeding node.*
 - b. *temp.next.prev = temp.prev; tells the succeeding node to link backwards past temp directly to the preceding node. This safely detaches temp from the chain.*
7. The **remove(int index)** method currently does not check for negative index or index that exceed the size of the DoubleLinkedLists. Please add statements to handle these cases.

```

158     void remove (int index)
159     {
160         if(isEmpty())
161         {
162             System.out.println(x: "Double linked list is currently empty!!");
163             return;
164         }
165
166         if (index < 0)
167         {
168             System.out.println(x: "Index cannot be negative!");
169             return;
170         }
171
172         if (index == 0)
173             removeFirst();
174         else
175         {
176             Node temp = head;
177             int i = 0;
178
179             for(i = 0; i < index && temp != null; i++)
180                 temp = temp.next;
181
182             if(temp == null)
183             {
184                 System.out.println(x: "Index out of bounds! Exceeds list size.");
185                 return;
186             }
187
188             if(temp == tail)
189                 removeLast();
190             else
191             {
192                 temp.prev.next = temp.next;
193                 temp.next.prev = temp.prev;
194             }
195         }
196     }

```

Github:

<https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet13>

--- *** ---